

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
ABAC	Attribute-Based Access Control	Kiểm soát truy cập dựa trên thuộc tính
aDSL	annotation-based Domain-Specific Language	Ngôn ngữ chuyên biệt miền dựa vào chú thích
AGL	Activity Graph Language	Ngôn ngữ đồ thị hoạt động
ASM	Abstract Syntax Meta-Model	Siêu mô hình cú pháp trừu tượng
AST	Abstract Syntax Tree	Cây cú pháp trừu tượng
CSM	Concrete Syntax Meta-Model	Siêu mô hình cú pháp cụ thể
CQRS	Command Query Responsibility Segregation	Phân tách trách nhiệm lệnh và truy vấn
DCSL	Domain Class Specification Language	Ngôn ngữ đặc tả lớp miền
DDD	Domain-Driven Design	Thiết kế hướng miền
DM	Domain Model	Mô hình miền
DSD	Dynamic Separation of Duty	Phân tách nhiệm vụ động
DSL	Domain-Specific Language	Ngôn ngữ chuyên biệt miền
M2M	Model to Model	Chuyển đổi mô hình sang mô hình
M2T	Model to Text	Chuyển đổi mô hình sang văn bản
MDE	Model-Driven Engineering	Kỹ nghệ hướng mô hình
MOSA	Module-based Software Architecture	Kiến trúc phần mềm dựa vào mô-đun
OCL	Object Constraint Language	Ngôn ngữ ràng buộc đối tượng
OMG	Object Management Group	Nhóm quản lý đối tượng
PA	Permission Assignment	Quan hệ gán quyền cho vai trò
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
RH	Role Hierarchy	Phân cấp vai trò
SoD	Separation of Duty	Phân tách nhiệm vụ
SSD	Static Separation of Duty	Phân tách nhiệm vụ tĩnh
UA	User Assignment	Quan hệ gán vai trò cho người dùng
UDML	Unified Domain Model Language	Ngôn ngữ mô hình miền hợp nhất
UL	Ubiquitous Language	Ngôn ngữ chung
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất

Mục lục

Danh mục các từ viết tắt	1
1 GIỚI THIỆU	6
1.1 Đặt vấn đề	6
1.1.1 Ví dụ thúc đẩy nghiên cứu: hệ thống CourseMan	6
1.1.2 Các thách thức nghiên cứu	7
1.1.3 Phát biểu bài toán nghiên cứu	9
1.2 Mục tiêu và phạm vi nghiên cứu	10
1.3 Nội dung và phương pháp nghiên cứu	11
1.4 Các đóng góp chính của luận văn	11
1.5 Cấu trúc luận văn	12
2 CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN TÌNH HÌNH NGHIÊN CỨU	13
2.1 Tổng quan về thiết kế hướng miền	13
2.1.1 Nền tảng thiết kế hướng miền	13
2.1.2 Mô hình miền hướng thực thi	14
2.2 Ngôn ngữ chuyên biệt miền	14
2.2.1 Khái niệm và phân loại	14
2.2.2 Phương pháp biểu diễn bằng siêu mô hình hóa	15
2.2.3 Ngôn ngữ chuyên biệt miền dựa vào chú thích và kiến trúc MOSA	15
2.3 Kiểm soát truy cập dựa trên vai trò	16
2.3.1 Mô hình cơ sở RBAC ₀	16
2.3.2 RBAC ₁ : phân cấp vai trò	16
2.3.3 RBAC ₂ : ràng buộc phân tách nhiệm vụ	16
2.3.4 Biểu diễn khía cạnh kiểm soát truy cập trong mô hình miền	17
2.4 Chuẩn mô hình hóa hình thức và công cụ	18
2.4.1 Cơ chế hồ sơ của UML	18
2.4.2 Ngôn ngữ ràng buộc đối tượng	18
2.4.3 Công cụ kiểm chứng USE	18
2.4.4 PlantUML: cú pháp văn bản của biểu đồ UML	18
2.5 Tích hợp mối quan tâm và chuyển đổi mô hình	19
2.5.1 Tích hợp các mối quan tâm vào mô hình miền hợp nhất	19

2.5.2	Các kỹ thuật chuyển đổi mô hình	19
2.5.3	Kiến trúc đích và nền tảng cài đặt	19
2.6	Tổng kết chương	20
3	PHƯƠNG PHÁP ĐỀ XUẤT	21
3.1	Tổng quan phương pháp	21
3.1.1	Nguyên tắc thiết kế	21
3.1.2	Kiến trúc tổng thể	21
3.2	Ngôn ngữ đặc tả chính sách RBAC	22
3.2.1	Cú pháp trừu tượng: siêu mô hình RBAC	22
3.2.2	Cú pháp cụ thể: profile trên biểu đồ lớp	23
3.2.3	Ngữ nghĩa tĩnh: tập bất biến	24
3.3	Bước 1 – Phân tích cú pháp	25
3.3.1	Mục tiêu và đầu ra	25
3.3.2	Xử lý từng nguồn đặc tả	26
3.3.3	Vị trí nguồn và chẩn đoán lỗi	26
3.4	Bước 2 – Hợp nhất mô hình	26
3.4.1	Mô hình miền hợp nhất	26
3.4.2	Tham chiếu chéo giữa các mối quan tâm	27
3.4.3	Bất biến sau hợp nhất	27
3.5	Bước 3 – Làm giàu mô hình	27
3.5.1	Ba phép biến đổi trên mối quan tâm cấu trúc	27
3.5.2	Lan truyền quyền theo phân cấp vai trò	28
3.6	Bước 4 – Kiểm chứng ràng buộc	29
3.6.1	Quy trình đánh giá	29
3.6.2	Ranh giới giữa hai thời điểm kiểm chứng	29
3.7	Bước 5 – Sinh mã theo mẫu	30
3.7.1	Ánh xạ khái niệm sang sản phẩm mã	30
3.7.2	Ba nguyên tắc của cơ chế sinh	30
3.8	Các tính chất của phương pháp	31
3.9	Tổng kết chương	31
4	THỰC NGHIỆM VÀ ĐÁNH GIÁ	32
4.1	Giới thiệu	32
4.2	Công cụ hỗ trợ	32
4.3	Ca nghiên cứu CourseMan	32
4.4	Thiết kế đánh giá	32
4.5	Kết quả thực nghiệm	32
4.6	Các nguy cơ ảnh hưởng tính hợp lệ	32
4.7	Bàn luận	32
4.8	Tổng kết chương	32

Danh sách hình vẽ

3.1 Tổng quan phương pháp ở mức khái niệm	22
3.2 Siêu mô hình chính sách RBAC (cú pháp trừu tượng)	23

Danh sách bảng

1.1	Đối sánh các nghiên cứu liên quan theo năm tiêu chí	9
2.1	Các khuôn mẫu chiến thuật của DDD được sử dụng trong luận văn	14
3.1	Ánh xạ giữa cú pháp trừu tượng và cú pháp cụ thể	24
3.2	Tập bất biến định nghĩa ngữ nghĩa tĩnh của ngôn ngữ	25
3.3	Ánh xạ khái niệm mô hình sang sản phẩm mã	30

Chương 1

GIỚI THIỆU

1.1 Đặt vấn đề

Phần mềm nghiệp vụ hiện đại phải xử lý những quy tắc ngày càng phức tạp, thay đổi liên tục và gắn chặt với ngữ cảnh của tổ chức sử dụng nó. Trước thực tế đó, thiết kế hướng miền (Domain-Driven Design – DDD) [3] đã nổi lên như một phương pháp phát triển phần mềm đặt mô hình miền (Domain Model – DM) vào vị trí trung tâm: thay vì bắt đầu từ cơ sở dữ liệu hay giao diện, đội phát triển cùng chuyên gia miền xây dựng một mô hình khái niệm phản ánh trung thực ngôn ngữ và quy tắc nghiệp vụ, rồi tổ chức toàn bộ mã nguồn xoay quanh mô hình đó. Các khái niệm nền tảng của DDD – ngôn ngữ chung (Ubiquitous Language – UL), ngữ cảnh giới hạn (bounded context) và các khuôn mẫu chiến thuật như thực thể, đối tượng giá trị, gốc tụ hợp, kho lưu trữ, dịch vụ miền, sự kiện miền – đã trở thành vốn từ chung của cộng đồng kỹ nghệ phần mềm [21].

Trong bối cảnh kỹ nghệ hướng mô hình (Model-Driven Engineering – MDE), nhiều nghiên cứu đã sử dụng ngôn ngữ chuyên biệt miền (Domain-Specific Language – DSL) để biểu diễn DM, qua đó liên kết chặt chẽ mô hình thiết kế với quá trình triển khai. Hướng nghiên cứu về ngôn ngữ chuyên biệt miền dựa vào chú thích (annotation-based DSL – aDSL) đã hình thành một mạch công trình nhất quán: DCSL [7, 8] biểu diễn mối quan tâm cấu trúc của DM; AGL [2, 10] biểu diễn mối quan tâm hành vi; UDML [6, 22] đề xuất cơ chế hợp nhất nhiều mối quan tâm vào một mô hình miền hợp nhất; và TMSA [9] xác định kiến trúc đích dạng vi dịch vụ phân tầng theo ngữ cảnh giới hạn. Điểm chung của mạch nghiên cứu này là: DM được đặc tả bằng một ngôn ngữ hình thức ở mức mô hình, sau đó bản mẫu phần mềm được sinh tự động từ đặc tả đó bằng các bộ chuyển đổi mô hình.

Luận văn này tiếp nối mạch nghiên cứu trên với một mối quan tâm đã được nhận diện nhưng chưa được khai thác đầy đủ ở mức công cụ: kiểm soát truy cập dựa trên vai trò (Role-Based Access Control – RBAC) [19].

1.1.1 Ví dụ thúc đẩy nghiên cứu: hệ thống CourseMan

Để làm rõ vấn đề nghiên cứu, luận văn sử dụng hệ thống quản lý đào tạo CourseMan làm ví dụ thúc đẩy xuyên suốt. Đây cũng là ca nghiên cứu điển hình đã được sử dụng

trong mạch công trình DCSL, AGL và UDML [7, 2, 22], nhờ đó kết quả của luận văn có thể đối sánh trực tiếp với các nghiên cứu trước.

CourseMan gồm hai ngữ cảnh giới hạn. Ngữ cảnh *ghi danh* xoay quanh gốc tụ hợp Student, cùng các thực thể CourseModule, Enrolment, các đối tượng giá trị StudentName, Grade, dịch vụ miễn EnrollmentService và hai ca sử dụng đăng ký sinh viên, ghi danh học phần. Ngữ cảnh *lớp học* xoay quanh gốc tụ hợp SClass và thực thể SClassRegistration.

Bên cạnh cấu trúc và hành vi, hệ thống có một chính sách kiểm soát truy cập với ba vai trò. Vai trò StudentRole chỉ được xem hồ sơ và bản ghi ghi danh của chính mình. Vai trò EnrollmentOfficer được tạo bản ghi ghi danh và xem hồ sơ sinh viên trong phạm vi phụ trách. Vai trò Admin kế thừa toàn bộ quyền của EnrollmentOfficer và được bổ sung quyền quản trị. Chính sách còn kèm ba loại ràng buộc: không ai đồng thời giữ vai trò Admin và vai trò kiểm toán (phân tách nhiệm vụ tĩnh); EnrollmentOfficer và StudentRole không được kích hoạt trong cùng một phiên làm việc (phân tách nhiệm vụ động); và số người giữ vai trò Admin không vượt quá một ngưỡng cho trước.

Ba câu hỏi thực tiễn nảy sinh từ ví dụ này, và cũng chính là ba vấn đề mà luận văn giải quyết:

1. Chính sách trên được *đặc tả ở đâu* để chuyên gia miền đọc hiểu và phê duyệt được, đồng thời máy xử lý được?
2. Làm thế nào *bảo đảm chính sách đó nhất quán* – không có vai trò xung khắc bị gán chồng, không có chu trình kế thừa, không có quyền trở tới khái niệm không tồn tại – *trước khi viết mã*?
3. Làm thế nào để mã nguồn thực thi *đúng bằng* chính sách đã phê duyệt, và giữ được sự tương ứng đó qua các lần thay đổi?

1.1.2 Các thách thức nghiên cứu

Kiểm soát truy cập là một mối quan tâm cắt ngang

RBAC [19] là mô hình kiểm soát truy cập được sử dụng rộng rãi nhất trong phần mềm doanh nghiệp: quyền không gán trực tiếp cho người dùng mà gán cho vai trò, người dùng nhận quyền thông qua quan hệ gán vai trò. Tuy phổ biến ở mức khái niệm, RBAC lại thường được cài đặt thủ công ở tầng hạ tầng: các đoạn mã kiểm tra quyền nằm rải rác trong bộ điều khiển, trong lớp dịch vụ, đôi khi cả trong DM.

Theo thuật ngữ của kỹ nghệ phần mềm, RBAC là một *mối quan tâm cắt ngang* (cross-cutting concern) điển hình, biểu hiện qua hai triệu chứng: *rải rác* (scattering) – lô-gic của cùng một chính sách bị phân tán ra nhiều mô-đun, và *trộn lẫn* (tangling) – câu lệnh kiểm tra quyền xen kẽ với lô-gic nghiệp vụ trong cùng một đơn vị mã. Hệ quả là chi phí bảo trì cao và rủi ro an toàn: một điểm kiểm tra bị bỏ sót trong hàng chục điểm rải rác đủ để tạo ra lỗ hổng, trong khi rà soát thủ công gần như không khả thi với hệ thống lớn.

Ba đặc thù khiến RBAC khó mô-đun hóa

So với các mối quan tâm cắt ngang quen thuộc như ghi nhật ký hay quản lý giao dịch, RBAC có ba đặc thù làm cho việc tách nó thành một mô-đun độc lập trở nên khó hơn

đáng kể.

Thứ nhất, tham chiếu chéo tới DM. Một quyền chỉ có nghĩa khi gắn với khái niệm nghiệp vụ cụ thể: quyền xem hồ sơ sinh viên chỉ được hiểu đúng khi biết nó cho phép thao tác *đọc* trên lớp miễn Student với phạm vi xác định. Nói cách khác, đặc tả kiểm soát truy cập buộc phải vượt ranh giới của chính nó để trở vào mối quan tâm cấu trúc.

Thứ hai, tính không cục bộ do phân cấp vai trò. Trong mô hình RBAC₁, vai trò cấp cao kế thừa toàn bộ quyền của vai trò cấp thấp hơn. Do đó tập quyền hiệu dụng của một vai trò không đọc được từ khai báo cục bộ mà phải tính qua bao đóng trên đồ thị kế thừa.

Thứ ba, ràng buộc phạm vi toàn hệ thống. Các ràng buộc phân tách nhiệm vụ tĩnh (SSD), phân tách nhiệm vụ động (DSD) và giới hạn số lượng được phát biểu trên toàn bộ tập vai trò và tập người dùng. Đây là các mệnh đề mà một điểm kiểm tra cục bộ trong mã nguồn không thể bảo đảm.

Khoảng cách giữa mô hình và mã nguồn

Thách thức thứ ba mang tính phương pháp luận. Trong thực tế, chính sách kiểm soát truy cập thường được mô tả trong tài liệu thiết kế bằng ngôn ngữ tự nhiên hoặc bảng biểu, rồi được lập trình viên hiện thực hóa thủ công. Không có cơ chế nào bảo đảm mã nguồn thực thi đúng chính sách đã phê duyệt, cũng không có cách nào kiểm chứng bản thân chính sách có nhất quán hay không trước khi viết mã. Sau vài chu kỳ phát triển, tài liệu thiết kế và mã nguồn phân kỳ; câu hỏi “ai thực sự được làm gì trong hệ thống này” chỉ trả lời được bằng cách đọc mã.

Khoảng trống trong các nghiên cứu liên quan

Bài toán nêu trên đã được tiếp cận từ nhiều góc độ khác nhau. Về nền tảng lý thuyết, Sandhu và cộng sự [19] định nghĩa họ mô hình tham chiếu RBAC₀–RBAC₃; Ray và cộng sự [17] biểu diễn ràng buộc RBAC bằng UML kết hợp OCL và trực quan hóa vi phạm bằng mẫu biểu đồ đối tượng; Sohr và cộng sự [20] phân tích và quản lý chính sách RBAC bằng OCL trên công cụ USE. Tuy vậy, các công trình này mô hình hóa RBAC như một hệ thống độc lập để phân tích, chưa gắn nó với DM của một ứng dụng cụ thể và không đề cập tới sinh mã.

Về an toàn hướng mô hình, SecureUML [11] và bản mở rộng [1] là những công trình mở đường: chính sách được mô hình hóa bằng một hồ sơ UML, sau đó hạ tầng kiểm soát truy cập được sinh tự động. Điểm khác biệt so với luận văn là SecureUML gắn với hệ công cụ mô hình hóa nặng của thời kỳ đó, không đặt trong khuôn khổ hợp nhất nhiều mối quan tâm của DDD, và không sinh ra bản mẫu phần mềm phân tầng theo ngữ cảnh giới hạn như kiến trúc hiện đại đòi hỏi.

Về mạch aDSL cho DDD, DCSL [7, 8] và AGL [2] đã cho thấy hiệu quả của việc đặc tả DM bằng chú thích rồi sinh mã, nhưng phạm vi của hai ngôn ngữ này không bao gồm kiểm soát truy cập. Gần đây, UDML [6, 22] đề xuất cơ chế hợp nhất nhiều mối quan tâm vào một DM hợp nhất và giới thiệu RBACDom – một DSL theo mối quan tâm bảo mật – với siêu mô hình cú pháp trừu tượng, ngữ nghĩa hình thức và cơ chế chú thích ở mức nút hành vi. Công trình này giải quyết bài toán *biểu diễn và hợp nhất* ở mức ngôn ngữ, nhưng chưa đi tới một quy trình công cụ hoàn chỉnh cho phép người thiết kế đặc

tả chính sách bằng ký pháp sơ đồ dạng văn bản quen thuộc, kiểm chứng chính sách đó, và sinh trực tiếp bản mẫu phần mềm có lớp thực thi kiểm soát truy cập đầy đủ trên một nền tảng hiện đại.

Bảng 1.1 đối sánh các nhóm công trình theo năm tiêu chí phản ánh trực tiếp yêu cầu của bài toán.

Bảng 1.1: Đối sánh các nghiên cứu liên quan theo năm tiêu chí

Công trình	Trọng tâm	(1)	(2)	(3)	(4)	(5)
Sandhu [19]	Mô hình tham chiếu RBAC	-	-	✓	-	-
Ray [17]	Biểu diễn ràng buộc UML/OCL	✓	-	✓	✓	-
Sohr [20]	Phân tích chính sách bằng OCL	✓	-	✓	✓	-
Basin [1]	An toàn hướng mô hình	✓	✓	-	-	✓
DCSL [7]	aDSL mối quan tâm cấu trúc	✓	✓	-	✓	✓
AGL [2]	aDSL mối quan tâm hành vi	✓	✓	-	✓	✓
UDML [22]	Hợp nhất mối quan tâm, RBACDom	✓	✓	✓	✓	✓
Luận văn	Quy trình công cụ hoàn chỉnh	✓	✓	✓	✓	✓

(1) Đặc tả ở mức mô hình (2) Liên kết với DM (3) Phân cấp vai trò và ràng buộc RBAC (4) Ngữ nghĩa hình thức kiểm chứng được (5) Sinh tự động bản mẫu phần mềm

1.1.3 Phát biểu bài toán nghiên cứu

Từ các phân tích trên, bài toán nghiên cứu của luận văn được phát biểu như sau.

Cho:

các đặc tả ở mức mô hình của một hệ thống phần mềm, gồm (i) biểu đồ lớp UML mô tả DM cùng với các vai trò, quyền và phân cấp vai trò; (ii) biểu đồ hoạt động UML mô tả hành vi của các ca sử dụng; và (iii) tập ràng buộc OCL đặc tả các bất biến của chính sách kiểm soát truy cập.

Tìm:

một phương pháp cùng công cụ hỗ trợ, chuyển đổi tự động các đặc tả trên thành một bản mẫu phần mềm phân tầng theo DDD kèm lớp thực thi kiểm soát truy cập đúng với chính sách đã đặc tả.

Sao cho:

(a) kiểm soát truy cập là một mối quan tâm hạng nhất của DM, ngang hàng với cấu trúc và hành vi; (b) tính trực giao được bảo toàn, nghĩa là bổ sung hoặc loại bỏ mối quan tâm kiểm soát truy cập không làm thay đổi hai mối quan tâm còn lại; (c) tồn tại quan hệ truy vết hai chiều giữa mô hình và mã nguồn; (d) toàn bộ quá trình tự động và lặp lại được.

Bài toán này được cụ thể hóa thành bốn câu hỏi nghiên cứu, được trả lời bằng thực nghiệm trong Chương 4:

RQ1. Tính khả thi: phương pháp và công cụ đề xuất có sinh được bản mẫu phần mềm hoạt động đúng với đặc tả RBAC đầu vào hay không?

- RQ2. Năng suất:** so với cài đặt thủ công, phương pháp tiết kiệm được bao nhiêu công sức đặc tả và bảo trì chính sách kiểm soát truy cập?
- RQ3. Tính đúng đắn:** các vi phạm ràng buộc trong đặc tả có được phát hiện và ngăn chặn trước khi sinh mã hay không?
- RQ4. Truy vết và mức dễ hiểu:** mã nguồn sinh ra có đối chiếu ngược được về phần tử mô hình tương ứng, và người đọc biểu đồ có nắm được chính sách kiểm soát truy cập của hệ thống hay không?

1.2 Mục tiêu và phạm vi nghiên cứu

Luận văn nghiên cứu và đề xuất phương pháp đặc tả, tích hợp mối quan tâm kiểm soát truy cập dựa trên vai trò vào DM theo tinh thần DDD, đồng thời xây dựng bộ chuyển đổi mô hình sinh tự động bản mẫu phần mềm từ đặc tả đó. Để đạt mục tiêu tổng quát này, luận văn tập trung vào ba mục tiêu cụ thể sau.

Mục tiêu 1. Nghiên cứu và đề xuất một DSL cho phép đặc tả chính sách RBAC ngay trong DM, được định nghĩa đầy đủ qua ba thành phần: siêu mô hình cú pháp trừu tượng, cú pháp cụ thể dạng văn bản trên biểu đồ lớp, và ngữ nghĩa tĩnh biểu diễn bằng tập bất biến OCL.

Mục tiêu 2. Đề xuất cơ chế tích hợp mối quan tâm kiểm soát truy cập vào DM hợp nhất cùng với mối quan tâm cấu trúc và hành vi trên một cây cú pháp trừu tượng duy nhất, bao gồm quan hệ tham chiếu chéo giữa quyền và lớp miễn, thuật toán lan truyền quyền theo phân cấp vai trò, và quy trình kiểm chứng bất biến ở giai đoạn thiết kế.

Mục tiêu 3. Nghiên cứu và xây dựng bộ chuyển đổi mô hình sinh tự động bản mẫu phần mềm phân tầng theo DDD kèm lớp thực thi kiểm soát truy cập, đồng thời bảo toàn quan hệ truy vết giữa mô hình và mã nguồn.

Phạm vi nghiên cứu của luận văn được giới hạn như sau:

- *Về mô hình kiểm soát truy cập:* hỗ trợ đầy đủ RBAC₀ và RBAC₁ bao gồm lan truyền quyền qua phân cấp vai trò; RBAC₂ được hỗ trợ ở mức đặc tả và kiểm chứng thiết kế. RBAC₃ và kiểm soát truy cập liên miễn giữa các ngữ cảnh giới hạn thuộc hướng phát triển.
- *Về ngôn ngữ ràng buộc:* hiện thực một tập con của OCL đủ để biểu diễn lớp bất biến RBAC₀-RBAC₂; các biểu thức ngoài tập con được xử lý bằng công cụ kiểm chứng hình thức bên ngoài.
- *Về nền tảng đích:* phương pháp được trình bày ở mức khái niệm, độc lập với công nghệ cài đặt; một nền tảng cụ thể được lựa chọn để minh chứng tính khả thi trong phần thực nghiệm.
- *Về đánh giá:* sử dụng ca nghiên cứu CourseMan; các giới hạn về khả năng khái quát hóa được phân tích trong phần nguy cơ ảnh hưởng tính hợp lệ.

1.3 Nội dung và phương pháp nghiên cứu

Để đạt được các mục tiêu đã nêu, luận văn thực hiện các nội dung nghiên cứu sau.

Nội dung 1 – Nghiên cứu tổng quan. Khảo sát các công trình về DDD, DSL và aDSL, mô hình RBAC, các chuẩn mô hình hóa UML/OCL, cơ chế hợp nhất mối quan tâm và các kỹ thuật chuyển đổi mô hình; từ đó xác định khoảng trống nghiên cứu và định vị đóng góp của luận văn.

Nội dung 2 – Thiết kế ngôn ngữ. Định nghĩa DSL đặc tả chính sách RBAC theo quy trình thiết kế ngôn ngữ chuẩn: xây dựng siêu mô hình cú pháp trừu tượng, thiết kế cú pháp cụ thể dưới dạng hồ sơ trên biểu đồ lớp, và đặc tả ngữ nghĩa tĩnh bằng các quy tắc hợp lệ cấu trúc phát biểu bằng OCL.

Nội dung 3 – Xây dựng bộ chuyển đổi mô hình. Thiết kế quy trình biến đổi năm bước gồm phân tích cú pháp, hợp nhất mô hình, làm giàu mô hình, kiểm chứng ràng buộc và sinh mã theo mẫu; trong đó bước làm giàu chứa thuật toán lan truyền quyền được chứng minh về tính đúng đắn và tính dừng.

Nội dung 4 – Hiện thực công cụ và thực nghiệm. Xây dựng công cụ hỗ trợ, áp dụng trên ca nghiên cứu CourseMan, đánh giá theo bốn câu hỏi nghiên cứu và phân tích các nguy cơ ảnh hưởng tính hợp lệ của kết quả.

Phương pháp nghiên cứu kết hợp nghiên cứu lý thuyết với nghiên cứu thực nghiệm. Về lý thuyết, luận văn sử dụng phương pháp siêu mô hình hóa để định nghĩa ngôn ngữ và sử dụng OCL để đặc tả ngữ nghĩa tĩnh. Về thực nghiệm, luận văn theo quy trình lặp lấy DM làm trung tâm: đặc tả, sinh mã, đánh giá và hiệu chỉnh, được kiểm chứng trên ca nghiên cứu điển hình.

1.4 Các đóng góp chính của luận văn

Luận văn có hai đóng góp chính, trong đó đóng góp thứ hai hiện thực hóa và kiểm chứng đóng góp thứ nhất.

- i. **Đề xuất phương pháp đặc tả và tích hợp mối quan tâm kiểm soát truy cập dựa trên vai trò vào DM.** Luận văn phát triển một DSL cho phép đặc tả chính sách RBAC ngay trong DM, gồm ba thành phần: siêu mô hình cú pháp trừu tượng của chính sách; cú pháp cụ thể dạng văn bản trên biểu đồ lớp với các khuôn dập vai trò, cú pháp khai báo quyền có ngữ nghĩa và các khuôn dập ràng buộc; và ngữ nghĩa tĩnh gồm tập quy tắc hợp lệ cấu trúc phát biểu bằng OCL. Kèm theo ngôn ngữ là cơ chế tích hợp: DM hợp nhất ba mối quan tâm trên một cây cú pháp trừu tượng, quan hệ tham chiếu chéo giữa quyền và lớp miễn, và thuật toán lan truyền quyền theo phân cấp vai trò bằng phép lặp điểm bất động.
- ii. **Xây dựng công cụ hỗ trợ và đánh giá thực nghiệm.** Luận văn hiện thực hóa phương pháp thành công cụ ddd-codegen theo quy trình biến đổi năm bước, sinh tự động bản mẫu phần mềm phân tầng theo DDD kèm lớp thực thi kiểm soát truy cập; đồng thời đánh giá trên ca nghiên cứu CourseMan theo bốn câu hỏi nghiên cứu, kèm phân tích các nguy cơ ảnh hưởng tính hợp lệ của kết quả.

Về quan hệ với các công trình của nhóm nghiên cứu, luận văn kế thừa trực tiếp cơ chế hợp nhất mối quan tâm của UDML và mối quan tâm bảo mật RBACDom [6, 22]. Điểm bổ sung của luận văn nằm ở ba khía cạnh: cú pháp cụ thể dạng văn bản cho phép đặc tả chính sách bằng công cụ biểu đồ phổ thông và quản lý bằng hệ thống quản lý phiên bản; quy trình kiểm chứng bất biến gắn liền với bước sinh mã, bảo đảm đặc tả sai không tạo ra mã; và bộ chuyển đổi mô hình hoàn chỉnh tới một nền tảng hiện đại kèm cơ chế truy vết hai chiều.

1.5 Cấu trúc luận văn

Phần còn lại của luận văn được tổ chức như sau.

Chương 2 trình bày cơ sở lý thuyết và tổng quan tình hình nghiên cứu, gồm: nền tảng DDD, ngôn ngữ chuyên biệt miền và hướng tiếp cận aDSL, mô hình RBAC, các chuẩn mô hình hóa UML/OCL, cơ chế hợp nhất mối quan tâm và kiến trúc đích.

Chương 3 trình bày phương pháp đề xuất: DSL đặc tả chính sách RBAC với ba thành phần cú pháp trừu tượng, cú pháp cụ thể và ngữ nghĩa tĩnh; quy trình biến đổi năm bước từ đặc tả tới mã nguồn; và phân tích các tính chất của phương pháp.

Chương 4 trình bày công cụ hỗ trợ, ca nghiên cứu, thiết kế đánh giá, kết quả thực nghiệm theo bốn câu hỏi nghiên cứu, các nguy cơ ảnh hưởng tính hợp lệ và bàn luận.

Phần **Kết luận** tổng kết đóng góp, nêu hạn chế và hướng nghiên cứu tiếp theo.

Chương 2

CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN TÌNH HÌNH NGHIÊN CỨU

Chương này trình bày cơ sở lý thuyết và tổng quan tình hình nghiên cứu liên quan tới luận văn. Mục 2.1 trình bày nền tảng thiết kế hướng miền. Mục 2.2 trình bày ngôn ngữ chuyên biệt miền và hướng tiếp cận dựa vào chú thích. Mục 2.3 trình bày mô hình kiểm soát truy cập dựa trên vai trò. Mục 2.4 trình bày các chuẩn mô hình hóa hình thức và công cụ kiểm chứng. Mục 2.5 khảo sát các hướng tiếp cận tích hợp mối quan tâm vào mô hình miền và các kỹ thuật chuyển đổi mô hình. Mục 2.6 tổng kết chương và định vị nghiên cứu của luận văn.

2.1 Tổng quan về thiết kế hướng miền

2.1.1 Nền tảng thiết kế hướng miền

Thiết kế hướng miền [3] xuất phát từ một quan sát: phần lớn độ phức tạp của phần mềm nghiệp vụ không nằm ở kỹ thuật mà ở bản thân miền vấn đề. Do đó, giá trị cốt lõi của thiết kế là xây dựng được một mô hình miền diễn đạt trung thực tri thức nghiệp vụ và được cả chuyên gia miền lẫn lập trình viên cùng hiểu.

Hai khái niệm chiến lược quan trọng nhất là ngôn ngữ chung và ngữ cảnh giới hạn. *Ngôn ngữ chung* là tập thuật ngữ thống nhất, được dùng đồng thời trong trao đổi, trong tài liệu và trong mã nguồn; khi một khái niệm nghiệp vụ đổi tên thì tên lớp trong mã cũng phải đổi theo. *Ngữ cảnh giới hạn* là ranh giới trong đó một DM và ngôn ngữ chung của nó có hiệu lực; cùng một từ có thể mang nghĩa khác nhau ở hai ngữ cảnh khác nhau, và ranh giới rõ ràng cho phép mỗi mô hình giữ được tính nhất quán nội tại.

Ngữ cảnh giới hạn có ý nghĩa đặc biệt đối với luận văn: nó vừa là đơn vị phân rã của DM, vừa là đơn vị phân rã của kiến trúc đích, và cũng chính là phạm vi hiệu lực tự nhiên của một tập vai trò trong chính sách kiểm soát truy cập.

2.1.2 Mô hình miền hướng thực thi

Ở mức chiến thuật, DDD cung cấp một bộ khuôn mẫu để cấu trúc DM [3, 21]. Bảng 2.1 tóm tắt các khuôn mẫu được luận văn sử dụng.

Bảng 2.1: Các khuôn mẫu chiến thuật của DDD được sử dụng trong luận văn

Khuôn mẫu	Ý nghĩa
Thực thể (entity)	Đối tượng có định danh riêng, tồn tại xuyên suốt vòng đời dù thuộc tính thay đổi.
Đối tượng giá trị (value object)	Đối tượng bất biến, được xác định bởi giá trị chứ không bởi định danh.
Gốc tụ hợp (aggregate root)	Thực thể đóng vai trò cửa ngõ của một tụ hợp, bảo đảm bất biến nghiệp vụ của cả tụ hợp; là đơn vị nhất quán của giao dịch.
Kho lưu trữ (repository)	Giao diện truy xuất tụ hợp, tách DM khỏi cơ chế lưu trữ cụ thể.
Dịch vụ miền (domain service)	Thao tác nghiệp vụ không thuộc riêng một thực thể nào, thường trải rộng trên nhiều tụ hợp.
Sự kiện miền (domain event)	Sự kiện có ý nghĩa nghiệp vụ đã xảy ra, dùng để truyền tin giữa các phần của hệ thống.

Khái niệm *mô hình miền hướng thực thi* nhấn mạnh rằng DM không chỉ là bản vẽ mô tả mà còn phải mang đủ thông tin để sinh ra phần mềm chạy được [7, 22]. Điều này đặt ra yêu cầu về độ giàu ngữ nghĩa của DM: ngoài cấu trúc, mô hình còn phải biểu diễn được hành vi, ràng buộc nghiệp vụ và – trong phạm vi luận văn này – cả chính sách kiểm soát truy cập.

Để giữ DM độc lập với hạ tầng, DDD khuyến nghị tổ chức mã nguồn theo bốn tầng [21]: *tầng miền* chứa lớp miền và quy tắc nghiệp vụ thuần túy; *tầng ứng dụng* điều phối các ca sử dụng; *tầng hạ tầng* hiện thực hóa các giao diện kỹ thuật; và *tầng trình bày* tiếp nhận yêu cầu từ bên ngoài. Vị trí của kiểm soát truy cập trong kiến trúc này là một quyết định thiết kế đáng lưu ý: điểm kiểm soát nên đặt ở tầng trình bày và tầng ứng dụng, còn tầng miền phải được giữ thuần khiết – đây chính là điều kiện bảo đảm tính trực giao giữa mối quan tâm kiểm soát truy cập và mối quan tâm cấu trúc.

2.2 Ngôn ngữ chuyên biệt miền

2.2.1 Khái niệm và phân loại

Ngôn ngữ chuyên biệt miền là ngôn ngữ máy tính có sức biểu đạt hạn chế, tập trung vào một miền cụ thể [4]. Đổi lại sự hạn chế đó, DSL cho phép diễn đạt bài toán bằng chính khái niệm của miền, ngắn gọn và dễ kiểm chứng hơn so với ngôn ngữ lập trình đa dụng.

Fowler [4] phân biệt *DSL nội sinh* được nhúng trong cú pháp của một ngôn ngữ chủ, và *DSL ngoại sinh* có cú pháp riêng cùng bộ phân tích riêng. Một khái niệm then chốt khác là *mô hình ngữ nghĩa* (semantic model): cấu trúc dữ liệu trung gian mà bộ phân tích tạo ra và bộ sinh mã tiêu thụ. Việc tách bạch mô hình ngữ nghĩa khỏi cú pháp cho phép thay đổi cú pháp mà không ảnh hưởng phần sinh mã, và ngược lại.

2.2.2 Phương pháp biểu diễn bằng siêu mô hình hóa

Voelter và cộng sự [23] hệ thống hóa quy trình xây dựng DSL quanh ba thành phần. Đây cũng là khung trình bày mà Chương 3 tuân theo, và phù hợp với cách các công trình của nhóm nghiên cứu định nghĩa DCSL, AGL và RBACDom [7, 2, 22].

1. **Cú pháp trừu tượng:** tập khái niệm của ngôn ngữ cùng quan hệ giữa chúng, được đặc tả bằng một *siêu mô hình* (Abstract Syntax Meta-Model – ASM). Đây là phần lõi của ngôn ngữ, độc lập với cách viết.
2. **Cú pháp cụ thể:** ký pháp mà người dùng thực sự viết hoặc vẽ, được đặc tả bằng siêu mô hình cú pháp cụ thể (Concrete Syntax Meta-Model – CSM). Một cú pháp trừu tượng có thể có nhiều cú pháp cụ thể, dạng văn bản hoặc dạng đồ họa.
3. **Ngữ nghĩa:** ý nghĩa của các cấu trúc ngôn ngữ, thường tách thành *ngữ nghĩa tĩnh* – các quy tắc hợp lệ mà một mô hình đúng đắn phải thỏa mãn, và *ngữ nghĩa động* – hành vi khi thực thi, thường được định nghĩa gián tiếp qua phép chuyển đổi sang một không gian ngữ nghĩa hình thức hoặc sang một ngôn ngữ đích.

Nhận xét 2.1. Cách phân chia này giải thích vì sao một DSL không đồng nghĩa với một bộ sinh mã. Bộ sinh mã chỉ hiện thực hóa ngữ nghĩa động; phần giá trị lâu dài của một DSL nằm ở siêu mô hình cú pháp trừu tượng và ngữ nghĩa tĩnh – những thành phần độc lập với công nghệ đích và tái sử dụng được khi nền tảng thay đổi.

2.2.3 Ngôn ngữ chuyên biệt miễn dựa vào chú thích và kiến trúc MOSA

Hướng tiếp cận aDSL [7] chọn cú pháp cụ thể là các chú thích (annotation) gắn trực tiếp lên phần tử của ngôn ngữ lập trình chủ. Ưu điểm là đặc tả nằm ngay cạnh mã nguồn và dùng lại được hệ công cụ sẵn có của ngôn ngữ chủ; đặc tả và mã không bao giờ nằm ở hai nơi tách rời.

DCSL [7, 8] là aDSL cho mối quan tâm cấu trúc, với ba khái niệm chính: DClass biểu diễn lớp miền cùng thuộc tính khả biến; DAttr biểu diễn thuộc tính miền cùng các ràng buộc như bắt buộc, độ dài, khóa định danh; và DAssoc biểu diễn liên kết giữa các lớp miền cùng loại liên kết và bội số. DCSL đi kèm kiến trúc phần mềm dựa vào mô-đun (Module-based Software Architecture – MOSA), trong đó mỗi lớp miền tương ứng một mô-đun phần mềm hoàn chỉnh.

AGL [2, 10] bổ sung mối quan tâm hành vi bằng khái niệm đồ thị hoạt động: mỗi ca sử dụng được mô tả bởi một chuỗi hành động có thứ tự, lấy từ biểu đồ hoạt động UML. Điểm quan trọng về mặt phương pháp là AGL không thay thế DCSL mà bổ sung vào cùng một DM – tiền đề trực tiếp cho ý tưởng hợp nhất nhiều mối quan tâm.

Nhận xét 2.2. Luận văn kế thừa tinh thần aDSL nhưng chọn cú pháp cụ thể dạng *văn bản trên biểu đồ lớp* thay vì chú thích trên mã nguồn. Lý do: chính sách kiểm soát truy cập là đối tượng cần được chuyên gia miền và cán bộ quản trị xem xét, nên một ký pháp trực quan có lợi thế hơn chú thích trong mã. Ánh xạ giữa chú thích trong ngôn ngữ chủ và khuôn dập trên biểu đồ là ánh xạ một-một, nên tinh thần aDSL được bảo toàn.

2.3 Kiểm soát truy cập dựa trên vai trò

2.3.1 Mô hình cơ sở RBAC₀

Sandhu và cộng sự [19] định nghĩa họ mô hình RBAC gồm bốn cấp. Mô hình cơ sở RBAC₀ được phát biểu như sau.

Định nghĩa 2.1 (RBAC₀). Mô hình RBAC₀ gồm các tập hợp U (người dùng), R (vai trò), P (quyền), S (phiên làm việc) cùng các quan hệ:

$$\begin{aligned} UA &\subseteq U \times R && \text{(gán vai trò cho người dùng)} \\ PA &\subseteq P \times R && \text{(gán quyền cho vai trò)} \\ user &: S \rightarrow U && \text{(chủ thể của phiên)} \\ roles &: S \rightarrow 2^R && \text{(các vai trò được kích hoạt trong phiên)} \end{aligned}$$

với ràng buộc $roles(s) \subseteq \{r \mid (user(s), r) \in UA\}$: người dùng chỉ có thể kích hoạt những vai trò đã được gán cho mình.

Điểm cốt lõi của RBAC₀ là lớp trung gian vai trò: quyền không gán trực tiếp cho người dùng. Nhờ đó, thay đổi nhân sự chỉ ảnh hưởng quan hệ UA , còn thay đổi chính sách chỉ ảnh hưởng quan hệ PA – hai loại thay đổi được tách bạch.

Trong mô hình gốc, quyền được xem là một phần tử trừu tượng của tập P . Luận văn tinh chỉnh khái niệm này bằng cách gán cho quyền một cấu trúc nội tại gồm tài nguyên được bảo vệ, thao tác và phạm vi, trong đó tài nguyên là một lớp của DM – đây chính là điểm nối giữa mối quan tâm kiểm soát truy cập và mối quan tâm cấu trúc, được trình bày chi tiết trong Chương 3.

2.3.2 RBAC₁: phân cấp vai trò

Định nghĩa 2.2 (RBAC₁). RBAC₁ mở rộng RBAC₀ bằng quan hệ thứ tự bộ phận $RH \subseteq R \times R$ trên tập vai trò. Ký hiệu $r_1 \succeq r_2$ nếu r_1 kế thừa r_2 . Khi đó tập quyền hiệu dụng của vai trò r là

$$perm^*(r) = \bigcup_{r' \preceq r} \{p \mid (p, r') \in PA\}.$$

Vai trò cấp cao thừa hưởng toàn bộ quyền của các vai trò cấp thấp hơn nó trong phân cấp. Quan hệ RH là thứ tự bộ phận nên phải phản xạ, bắc cầu và phản đối xứng; hệ quả thực tiễn quan trọng là đồ thị kế thừa vai trò không được chứa chu trình. Bất biến này được luận văn kiểm tra tường minh ở bước kiểm chứng, vì một chu trình sẽ khiến phép tính $perm^*$ mất ý nghĩa thứ tự: mọi vai trò trong chu trình sẽ có cùng tập quyền, tức phân cấp trở nên vô nghĩa.

2.3.3 RBAC₂: ràng buộc phân tách nhiệm vụ

RBAC₂ bổ sung các ràng buộc trên những quan hệ đã có. Ba nhóm ràng buộc được luận văn hỗ trợ như sau.

Phân tách nhiệm vụ tĩnh (Static Separation of Duty – SSD): hai vai trò xung khắc không được gán đồng thời cho cùng một người dùng,

$$\forall u \in U : \neg((u, r_1) \in UA \wedge (u, r_2) \in UA) \quad \text{với } (r_1, r_2) \in SSD.$$

Ràng buộc này quyết định được ngay ở mức mô hình vì chỉ phụ thuộc quan hệ UA .

Phân tách nhiệm vụ động (Dynamic Separation of Duty – DSD): hai vai trò xung khắc có thể cùng được gán nhưng không được kích hoạt trong cùng một phiên,

$$\forall s \in S : \neg(r_1 \in \text{roles}(s) \wedge r_2 \in \text{roles}(s)) \quad \text{với } (r_1, r_2) \in DSD.$$

Vì phụ thuộc trạng thái phiên nên ràng buộc này chỉ kiểm tra được lúc vận hành.

Giới hạn số lượng (cardinality): số người dùng được gán một vai trò không vượt quá ngưỡng cho trước, $|\{u \mid (u, r) \in UA\}| \leq \max(r)$.

Nhận xét 2.3. Sự khác biệt giữa SSD và DSD minh họa một nguyên tắc thiết kế xuyên suốt luận văn: *ràng buộc nào quyết định được từ mô hình thì kiểm chứng ở giai đoạn thiết kế; ràng buộc nào phụ thuộc trạng thái vận hành thì phải sinh thành điểm kiểm tra trong mã nguồn.* Nguyên tắc này được cụ thể hóa ở bước kiểm chứng của phương pháp trong Chương 3.

RBAC₃ hợp nhất phân cấp vai trò với ràng buộc, làm nảy sinh các tương tác tinh tế – chẳng hạn hai vai trò xung khắc có thể cùng bị kế thừa bởi một vai trò cấp trên. Luận văn không xử lý cấp này và xếp vào hướng nghiên cứu tiếp theo.

2.3.4 Biểu diễn khía cạnh kiểm soát truy cập trong mô hình miền

Ray và cộng sự [17] đề xuất biểu diễn chính sách RBAC bằng UML kết hợp OCL: các khái niệm vai trò, quyền, phiên được mô hình hóa bằng lớp UML, còn các ràng buộc RBAC₂ được phát biểu bằng bất biến OCL. Đáng góp đáng chú ý là ý tưởng *mẫu vi phạm*: mỗi bất biến đi kèm một mẫu biểu đồ đối tượng mô tả tình huống vi phạm, giúp trực quan hóa lỗi cho người thiết kế. Sohr và cộng sự [20] phát triển hướng này thành quy trình phân tích và quản lý chính sách RBAC bằng OCL trên công cụ USE.

Basin và cộng sự [11, 1] đi xa hơn về phía sinh mã với SecureUML: chính sách được đặc tả bằng một hồ sơ UML và hạ tầng kiểm soát truy cập được sinh tự động. Đây là công trình gần nhất với luận văn về tinh thần; điểm khác biệt đã được phân tích ở mục 1.1.2.

Trong mạch nghiên cứu của nhóm, RBACDom [22] được đề xuất như một DSL theo mối quan tâm bảo mật trong khuôn khổ UDML. RBACDom định nghĩa siêu mô hình nắm bắt các khái niệm RBAC chuẩn (người dùng, vai trò, quyền, phiên), biểu diễn quyền dưới dạng cặp *thao tác* và *tài nguyên được bảo vệ*, mô hình hóa tường minh ràng buộc phân tách nhiệm vụ qua siêu khái niệm trừu tượng SoDConstraint cùng hai chuyên biệt hóa SSD và DSD, đồng thời cung cấp chiến lược kết hợp luật để xử lý nhiều chú thích cùng áp dụng trên một nút hành vi. Luận văn kế thừa trực tiếp cách tổ chức khái niệm này và bổ sung phần cú pháp cụ thể dạng văn bản cùng bộ chuyển đổi mô hình tới nền tảng thực thi.

Ngoài RBAC, các mô hình kiểm soát truy cập khác như kiểm soát truy cập dựa trên thuộc tính (ABAC) cho phép biểu đạt chính sách linh hoạt hơn nhưng đánh đổi bằng độ phức tạp của việc phân tích và kiểm chứng. Luận văn giới hạn phạm vi ở RBAC vì đây là

mô hình phổ biến nhất trong phần mềm nghiệp vụ và có ngữ nghĩa hình thức đã được chuẩn hóa.

2.4 Chuẩn mô hình hóa hình thức và công cụ

2.4.1 Cơ chế hồ sơ của UML

UML [15] cung cấp cơ chế mở rộng nhẹ gọi là *hồ sơ* (profile): thay vì sửa đổi siêu mô hình chuẩn, người thiết kế định nghĩa các *khuôn dập* (stereotype) gắn lên phần tử mô hình để bổ sung ngữ nghĩa chuyên biệt cho một miền. Đây là cách chuẩn hóa để một cộng đồng định nghĩa phương ngữ UML riêng mà vẫn dùng được công cụ UML thông thường. Luận văn định nghĩa cú pháp cụ thể của ngôn ngữ đặc tả chính sách RBAC theo đúng cơ chế này.

2.4.2 Ngôn ngữ ràng buộc đối tượng

OCL [14] là ngôn ngữ khai báo, không có tác dụng phụ, dùng để phát biểu ràng buộc trên mô hình UML. Các cấu trúc được luận văn sử dụng gồm: khai báo bất biến trong một ngữ cảnh (context...inv), các phép trên tập hợp (forAll, exists, includes, isUnique), phép duyệt quan hệ và các biểu thức lô-gic. Listing 2.1 minh họa hai bất biến RBAC viết bằng OCL.

```
context Role
  inv UniqueRoleName:
    Role.allInstances()->isUnique(name)

context User
  inv OnlyAssignedRolesActivated:
    self.sessions->forAll(s |
      s.activeRoles->forAll(r | self.assignedRoles->includes(r)))
```

Listing 2.1: Hai bất biến RBAC viết bằng OCL

2.4.3 Công cụ kiểm chứng USE

USE [5] là môi trường đặc tả và kiểm chứng mô hình UML/OCL: người dùng khai báo mô hình cùng các bất biến, tạo các ảnh chụp trạng thái dưới dạng biểu đồ đối tượng, và công cụ báo bất biến nào bị vi phạm. USE được luận văn xác định là công cụ cho bước kiểm chứng hình thức ở giai đoạn thiết kế trong hướng hoàn thiện.

2.4.4 PlantUML: cú pháp văn bản của biểu đồ UML

PlantUML [16] cho phép mô tả biểu đồ UML bằng văn bản thuần và kết xuất thành hình vẽ. So với các công cụ mô hình hóa đồ họa, ký pháp văn bản có ba ưu điểm quan trọng đối với luận văn: quản lý được bằng hệ thống quản lý phiên bản như mã nguồn; so sánh khác biệt giữa hai phiên bản dễ dàng; và tự động hóa được trong quy trình tích hợp liên

tục. Hạn chế của PlantUML là không có siêu mô hình hình thức đi kèm – đây chính là lý do luận văn phải tự định nghĩa siêu mô hình cú pháp trừu tượng riêng trong Chương 3.

2.5 Tích hợp mối quan tâm và chuyển đổi mô hình

2.5.1 Tích hợp các mối quan tâm vào mô hình miền hợp nhất

UDML [6, 22] giải quyết bài toán: làm thế nào để nhiều mối quan tâm cùng tồn tại trong một DM hợp nhất mà không cản trở nhau. Giải pháp dựa trên hai khái niệm: *phần tử có thể gắn chú thích* (Annotable) và *chú thích* (Annotation) – đơn vị thông tin thuộc về một mối quan tâm, gắn vào một phần tử và giữ tham chiếu ngược tới phần tử đó. Nhờ vậy, các mối quan tâm có thể tham chiếu chéo lẫn nhau ngay trong cây cú pháp trừu tượng.

Ba tính chất mà một phép hợp nhất đúng đắn phải bảo đảm là: *tính trực giao* – bổ sung hoặc loại bỏ một mối quan tâm không làm thay đổi ngữ nghĩa của các mối quan tâm còn lại; *tính gắn kết* – mọi tham chiếu chéo đều phân giải được về một phần tử tồn tại; và *tính truy vết* – mỗi phần tử của DM hợp nhất giữ được liên kết về vị trí xuất xứ trong đặc tả nguồn. Ba tính chất này được luận văn sử dụng làm tiêu chí thiết kế cho bước hợp nhất mô hình và được kiểm tra lại trong phần đánh giá.

2.5.2 Các kỹ thuật chuyển đổi mô hình

Trong kỹ nghệ hướng mô hình, hai loại chuyển đổi được sử dụng phổ biến: chuyển đổi mô hình sang mô hình (M2M) biến một mô hình nguồn thành mô hình đích theo một tập luật, và chuyển đổi mô hình sang văn bản (M2T) sinh chế tác văn bản, thường là mã nguồn, từ mô hình [22]. Phương pháp của luận văn sử dụng cả hai loại: các bước hợp nhất và làm giàu mô hình thuộc loại M2M, còn bước sinh mã thuộc loại M2T dựa trên mẫu.

Cơ chế sinh mã dựa trên mẫu đòi hỏi một nguyên tắc kỷ luật: mẫu sinh nên giữ ở dạng *không chứa lô-gic*, mọi tính toán được thực hiện trước trong bước chuẩn bị dữ liệu. Nhờ đó lô-gic sinh mã kiểm thử được bằng kiểm thử đơn vị thông thường thay vì phải so khớp văn bản kết xuất.

2.5.3 Kiến trúc đích và nền tảng cài đặt

Kiến trúc TMSA [9] đề xuất tổ chức các dịch vụ của một hệ thống theo cấu trúc đa cây: mỗi dịch vụ là một cây các mô-đun hướng miền, các cạnh liên cây cho phép tái sử dụng cấu trúc giữa các dịch vụ. Siêu mô hình của TMSA được đặc tả bằng UML/OCL, cho phép định nghĩa hình thức các mẫu chịu lỗi và các phép tiến hóa kiến trúc. Đối với luận văn, TMSA cung cấp định hướng về cấu trúc đầu ra: mỗi ngữ cảnh giới hạn tương ứng một mô-đun dịch vụ phân tầng, và điểm kiểm soát truy cập được đặt tại ranh giới của mô-đun đó. Các nguyên tắc thiết kế vi dịch vụ tổng quát hơn được trình bày trong [13, 18].

Về nền tảng cài đặt, luận văn chọn NestJS [12] – khung phát triển ứng dụng phía máy chủ trên nền Node.js với kiến trúc mô-đun rõ ràng. Ba cơ chế của nền tảng này được bước sinh mã sử dụng trực tiếp: *bộ bảo vệ* (guard) chặn yêu cầu trước khi tới bộ xử lý,

là nơi tự nhiên để đặt điểm kiểm soát truy cập; *chú thích tùy biến* và *phản chiếu siêu dữ liệu* cho phép gắn thông tin vai trò, quyền yêu cầu lên từng điểm cuối và đọc lại lúc chạy; và *mô-đun CQRS* tách lệnh khỏi truy vấn, phù hợp với cách AGL mô tả ca sử dụng như một chuỗi hành động. Việc chọn TypeScript làm ngôn ngữ đích cũng có lý do về mặt phương pháp: chú thích tùy biến của TypeScript là tương đương trực tiếp của chú thích trong Java, nên tinh thần aDSL của mạch DCSL/AGL được bảo toàn khi chuyển sang hệ sinh thái này.

2.6 Tổng kết chương

Chương này đã trình bày cơ sở lý thuyết và tổng quan tình hình nghiên cứu của luận văn. Từ DDD, luận văn kế thừa khái niệm DM, ngữ cảnh giới hạn, các khuôn mẫu chiến thuật và kiến trúc phân tầng. Từ lý thuyết DSL, luận văn kế thừa khung ba thành phần – cú pháp trừu tượng, cú pháp cụ thể, ngữ nghĩa – làm bố cục trình bày cho ngôn ngữ đề xuất, cùng tinh thần aDSL của DCSL và AGL. Từ RBAC, luận văn kế thừa ngữ nghĩa hình thức của các cấp $RBAC_0$ – $RBAC_2$ (Định nghĩa 2.1 và Định nghĩa 2.2) làm cơ sở cho siêu mô hình chính sách và cho thuật toán lan truyền quyền. Từ UML/OCL, luận văn kế thừa cơ chế hỗ sơ cho cú pháp cụ thể và OCL cho ngữ nghĩa tĩnh. Từ UDML và RBACDom, luận văn kế thừa cơ chế hợp nhất mối quan tâm cùng ba tính chất trực giao, gắn kết, truy vết, và cách tổ chức khái niệm của mối quan tâm bảo mật.

Khoảng trống mà các công trình trên còn để lại – và cũng là nội dung của chương tiếp theo – là một quy trình hoàn chỉnh cho phép đặc tả chính sách RBAC bằng ký pháp văn bản trên biểu đồ lớp, kiểm chứng chính sách đó ở giai đoạn thiết kế, và sinh tự động bản mẫu phần mềm có lớp thực thi kiểm soát truy cập đầy đủ, kèm quan hệ truy vết hai chiều giữa mô hình và mã nguồn.

Chương 3

PHƯƠNG PHÁP ĐỀ XUẤT

Chương này trình bày đóng góp chính của luận văn. Mục 3.1 phác thảo toàn cảnh phương pháp ở mức khái niệm. Mục 3.2 định nghĩa ngôn ngữ đặc tả chính sách RBAC theo ba thành phần đã nêu ở mục 2.2.2. Năm mục tiếp theo (3.3–3.7) trình bày chi tiết từng bước của quy trình biến đổi. Mục 3.8 phân tích các tính chất mà phương pháp bảo đảm.

3.1 Tổng quan phương pháp

3.1.1 Nguyên tắc thiết kế

Phương pháp được xây dựng trên bốn nguyên tắc, mỗi nguyên tắc trả lời trực tiếp một yêu cầu đã nêu trong phát biểu bài toán ở mục 1.1.3:

1. **Một nguồn sự thật duy nhất.** Chính sách kiểm soát truy cập chỉ được khai báo ở một nơi – mô hình – và mọi biểu hiện khác của nó (mã nguồn, cấu hình, tài liệu) đều được sinh ra từ đó. Nguyên tắc này loại trừ khả năng mô hình và mã phân kỳ.
2. **Kiểm soát truy cập là mối quan tâm hàng nhất.** Vai trò, quyền và ràng buộc là các phần tử của mô hình miền, có siêu mô hình riêng và tham chiếu trực tiếp tới lớp miền, chứ không phải siêu dữ liệu phụ trợ gắn ngoài.
3. **Kiểm chứng trước khi sinh.** Mô hình phải vượt qua bước kiểm chứng bất biến thì mới được chuyển sang bước sinh mã. Một đặc tả sai không bao giờ tạo ra mã.
4. **Độc lập với công nghệ cài đặt.** Bốn bước đầu chỉ thao tác trên các khái niệm mô hình hóa; mọi tri thức về nền tảng đích được cô lập trong bộ mẫu sinh của bước cuối.

3.1.2 Kiến trúc tổng thể

Phương pháp có dạng một quy trình biến đổi một chiều gồm năm bước, chuyển đặc tả ở mức mô hình thành phần mềm thực thi. Hình 3.1 minh họa đầu vào, các bước xử lý, dữ liệu trung gian và đầu ra.

Đầu vào gồm ba tài liệu đặc tả bổ sung cho nhau:

[Hình 3.1 – Sơ đồ tổng quan phương pháp: ba tài liệu đặc tả đầu vào → năm bước xử lý → bốn sản phẩm dữ liệu trung gian → bản mẫu phần mềm phân tầng kèm lớp thực thi kiểm soát truy cập, kèm vòng phản hồi từ bước kiểm chứng về đặc tả]

Hình 3.1: Tổng quan phương pháp ở mức khái niệm

- *Biểu đồ lớp* mô tả đồng thời cấu trúc miền (các lớp, thuộc tính, quan hệ) và chính sách kiểm soát truy cập (vai trò, quyền, phân cấp, ràng buộc). Việc đặt hai khía cạnh trên cùng một sơ đồ là chủ ý: nó cho phép quyền tham chiếu trực tiếp tới lớp miền mà không cần cơ chế liên kết ngoài.
- *Biểu đồ hoạt động* mô tả hành vi của từng ca sử dụng dưới dạng chuỗi hành động, theo ngôn ngữ AGL.
- *Tập ràng buộc* chứa các bất biến OCL – phần ngữ nghĩa hình thức mà ký pháp sơ đồ không diễn đạt hết.

Năm bước xử lý lần lượt là: phân tích cú pháp, hợp nhất mô hình, làm giàu mô hình, kiểm chứng ràng buộc và sinh mã theo mẫu.

Bốn sản phẩm dữ liệu trung gian nằm giữa các bước: cây cú pháp của từng nguồn, mô hình miền hợp nhất, mô hình đã làm giàu, và mô hình hợp lệ. Việc đặt tên và định nghĩa rõ từng sản phẩm trung gian không chỉ nhằm trình bày: nó cho phép mỗi bước được kiểm thử độc lập, và cho phép chèn thêm bước mới trong tương lai mà không phá vỡ các bước hiện có.

Vòng phản hồi đi từ bước kiểm chứng ngược về đặc tả: khi phát hiện vi phạm, quy trình dừng lại và trả về chẩn đoán trở tới vị trí gây lỗi trong tài liệu đặc tả gốc, người thiết kế hiệu chỉnh rồi chạy lại. Đây là điểm khác biệt cơ bản so với các bộ sinh mã thông thường, vốn chỉ kiểm tra tính hợp lệ cú pháp chứ không kiểm chứng ngữ nghĩa của chính sách.

Nhận xét 3.1. Toàn bộ nội dung mục 3.1 không nhắc tới bất kỳ ngôn ngữ lập trình hay khung phần mềm nào. Đây là chủ ý: phương pháp được phát biểu ở mức khái niệm, và việc chọn một nền tảng đích cụ thể chỉ diễn ra ở bước 5 và ở phần thực nghiệm.

3.2 Ngôn ngữ đặc tả chính sách RBAC

Mục này trình bày đóng góp thứ nhất của luận văn theo đúng ba thành phần của một ngôn ngữ: cú pháp trừu tượng, cú pháp cụ thể và ngữ nghĩa tĩnh.

3.2.1 Cú pháp trừu tượng: siêu mô hình RBAC

Cú pháp trừu tượng được đặc tả bằng một siêu mô hình gồm bảy khái niệm. Định nghĩa 3.1 phát biểu hình thức cấu trúc này.

Định nghĩa 3.1 (Siêu mô hình chính sách RBAC). Một chính sách kiểm soát truy cập là bộ

$$\mathcal{P} = \langle \mathcal{R}, \mathcal{Q}, \text{grant}, \text{extends}, \mathcal{C}_{ssd}, \mathcal{C}_{dsd}, \text{card} \rangle$$

trong đó:

- $\mathcal{R}$ là tập hữu hạn các *vai trò*, mỗi vai trò có tên duy nhất;
- $\mathcal{Q}$ là tập hữu hạn các *quyền*; mỗi quyền $q \in \mathcal{Q}$ là bộ ba $\langle \text{res}(q), \text{act}(q), \text{scope}(q) \rangle$ với $\text{res}(q)$ là một lớp của mô hình miền, $\text{act}(q)$ là một thao tác và $\text{scope}(q)$ là phạm vi áp dụng;
- $\text{grant} \subseteq \mathcal{R} \times \mathcal{Q}$ là quan hệ gán quyền cho vai trò (tương ứng quan hệ PA trong Định nghĩa 2.1);
- $\text{extends} \subseteq \mathcal{R} \times \mathcal{R}$ là quan hệ kế thừa vai trò (tương ứng RH trong Định nghĩa 2.2);
- $\mathcal{C}_{ssd}, \mathcal{C}_{dsd} \subseteq \mathcal{R} \times \mathcal{R}$ là các tập cặp vai trò xung khắc tĩnh và động;
- $\text{card} : \mathcal{R} \rightarrow \mathbb{N}$ là hàm bộ phận gán giới hạn số người giữ vai trò.

Điểm cần nhấn mạnh trong Định nghĩa 3.1 là thành phần $\text{res}(q)$: nó không phải một chuỗi ký tự tự do mà là *tham chiếu tới một lớp của mô hình miền*. Chính thành phần này biến chính sách kiểm soát truy cập từ một danh sách nhả rời rạc thành một phần gắn kết của mô hình, và cũng chính nó đặt ra yêu cầu về tính gắn kết sẽ được kiểm tra ở bước 4.

[Hình 3.2 – Siêu mô hình chính sách RBAC dạng biểu đồ lớp: Role, Permission, Resource, Action, Scope, RoleHierarchy, SsdConstraint, DsdConstraint, Cardinality và quan hệ giữa chúng]

Hình 3.2: Siêu mô hình chính sách RBAC (cú pháp trừu tượng)

3.2.2 Cú pháp cụ thể: profile trên biểu đồ lớp

Cú pháp cụ thể được định nghĩa dưới dạng một hồ sơ UML theo cơ chế khuôn dập đã trình bày ở mục 2.4. Bảng 3.1 là ánh xạ đầy đủ giữa khái niệm siêu mô hình và ký pháp trên sơ đồ.

Cấu trúc nội tại của biểu thức quyền được đặc tả bằng văn phạm sau:

```
Permission ::= Name ":" Resource "." Action [ "[" Scope "]" ]
Name       ::= UPPER_IDENT
Resource   ::= IDENT          (* ten mot lop mien tren cung so do *)
Action     ::= "create" | "read" | "update" | "delete" | IDENT
Scope      ::= "all" | "own" | IDENT
```

Listing 3.1: Văn phạm của biểu thức quyền (ký pháp EBNF)

Ba quyết định thiết kế trong văn phạm này cần được lý giải:

Bảng 3.1: Ánh xạ giữa cú pháp trừu tượng và cú pháp cụ thể

Khái niệm	Ký pháp trên sơ đồ	Ghi chú
Vai trò	class R <<Role>>	Tên lớp là tên vai trò
Quyền	Trường trong thân lớp vai trò, dạng TEN : TaiNguyen.thaoTac [phamVi]	Xem văn phạm 3.1
Gán quyền	Quyền được khai báo bên trong lớp vai trò	Quan hệ <i>grant</i> suy ra từ vị trí khai báo
Kế thừa vai trò	R1 -- > R2	R_1 kế thừa R_2
Phân phân tách nhiệm vụ vụ tĩnh	R1 ..> R2 : <<SSD>>	Cặp không đối xứng về ký pháp, đối xứng về ngữ nghĩa
Phân phân tách nhiệm vụ vụ động	R1 ..> R2 : <<DSD(maxActiveRoles=n)>>	
Giới hạn số lượng	<<cardinality max=n>> trong thân lớp vai trò	
Nhóm chính sách	package "X" <<AccessControl>>	Tách chính sách khỏi phần cấu trúc trên cùng sơ đồ

- **Vì sao tách Resource và Action thay vì dùng một tên quyền tự do?** Vì chỉ khi tách bạch, hệ thống mới kiểm tra được rằng tài nguyên được nhắc tới thực sự tồn tại trong mô hình miền, và mới ánh xạ được tự động từ quyền sang các điểm cuối thao tác trên tài nguyên đó ở bước sinh mã.
- **Vì sao Scope là tùy chọn?** Để giữ ngưỡng sử dụng thấp: phần lớn quyền không cần phạm vi, và người thiết kế chỉ khai báo phạm vi khi nó thực sự tạo khác biệt (ví dụ phân biệt *đọc mọi hồ sơ* với *đọc hồ sơ của chính mình*).
- **Vì sao Action cho phép cả định danh tự do lẫn bốn thao tác chuẩn?** Bốn thao tác CRUD phủ phần lớn nhu cầu và ánh xạ được tự động sang các thao tác mặc định sinh ra ở bước 3; định danh tự do dành cho các thao tác nghiệp vụ đặc thù, khi đó tên thao tác phải trùng với tên một phương thức hoặc ca sử dụng đã khai báo.

Nhận xét 3.2. Việc chọn biểu đồ lớp làm nơi khai báo chính sách – thay vì một tệp cấu hình riêng – có một hệ quả đáng giá: chính sách kiểm soát truy cập trở thành đối tượng có thể *xem xét bằng mắt* cùng lúc với mô hình miền. Chuyên gia nghiệp vụ nhìn vào một sơ đồ duy nhất là biết vai trò nào chạm được vào khái niệm nghiệp vụ nào.

3.2.3 Ngữ nghĩa tĩnh: tập bất biến

Không phải mọi mô hình viết đúng cú pháp đều là một chính sách hợp lệ. Ngữ nghĩa tĩnh của ngôn ngữ được đặc tả bằng tập bất biến trong Bảng 3.2, phân thành ba lớp theo cấp của mô hình RBAC.

Mỗi bất biến được phát biểu hình thức bằng OCL. Listing 3.2 trình bày ba bất biến tiêu biểu.

```
-- I1: ten vai tro la duy nhat
context Role
```

Bảng 3.2: Tập bất biến định nghĩa ngữ nghĩa tĩnh của ngôn ngữ

Mã	Lớp	Phát biểu	Thời điểm
I1	RBAC ₀	Tên vai trò là duy nhất trong toàn chính sách	Thiết kế
I2	RBAC ₀	Mọi quyền đều có tài nguyên và thao tác xác định	Thiết kế
I3	RBAC ₀	Tài nguyên của mọi quyền phải là một lớp tồn tại trong mô hình miền	Thiết kế
I4	RBAC ₀	Thao tác tự do của quyền phải trùng tên một phương thức hoặc ca sử dụng đã khai báo	Thiết kế
I5	RBAC ₁	Đồ thị kế thừa vai trò không chứa chu trình	Thiết kế
I6	RBAC ₂	Hai vai trò xung khắc tĩnh không được gán cho cùng một người dùng	Thiết kế & vận hành
I7	RBAC ₂	Hai vai trò xung khắc động không được kích hoạt trong cùng một phiên	Vận hành
I8	RBAC ₂	Số người giữ một vai trò không vượt quá giới hạn khai báo	Vận hành

```

inv UniqueRoleName:
    Role.allInstances()->isUnique(name)

-- I3: tai nguyen cua quyen phai la mot lop mien ton tai
context Permission
    inv ResourceExists:
        DomainClass.allInstances()->exists(c | c.name = self.resource)

-- I5: do thi ke thua vai tro khong co chu trinh
context Role
    inv AcyclicHierarchy:
        not self.allParents()->includes(self)

```

Listing 3.2: Đặc tả OCL của các bất biến I1, I3 và I5

Cột “Thời điểm” của Bảng 3.2 thể hiện nguyên tắc đã nêu ở Nhận xét 2.3: bất biến nào quyết định được từ mô hình thì kiểm ở thời điểm thiết kế; bất biến phụ thuộc trạng thái vận hành (phiên làm việc, số người dùng thực tế) buộc phải sinh thành điểm kiểm tra trong mã. Riêng I6 xuất hiện ở cả hai cột: phép gán vai trò khai báo tĩnh trong đặc tả kiểm được lúc thiết kế, còn phép gán phát sinh lúc hệ thống đang chạy phải kiểm lúc vận hành.

3.3 Bước 1 - Phân tích cú pháp

3.3.1 Mục tiêu và đầu ra

Bước 1 chuyển ba tài liệu đặc tả dạng văn bản thành cấu trúc dữ liệu mà máy xử lý được. Đầu ra là ba cây cú pháp độc lập, mỗi cây phản ánh trung thực nội dung của một nguồn, chưa có liên kết chéo giữa chúng.

Nguyên tắc thiết kế của bước này là *trung thực và không suy diễn*: bộ phân tích chỉ ghi nhận những gì được viết ra, không tự bổ sung phần tử nào. Mọi phép suy diễn được

đẩy sang bước 3, nơi chúng được thực hiện tập trung và kiểm thử được.

3.3.2 Xử lý từng nguồn đặc tả

Biểu đồ lớp. Bộ phân tích duyệt tuần tự tài liệu, nhận diện lần lượt: khối gói kèm khuôn đập ngữ cảnh giới hạn; khai báo lớp kèm khuôn đập; các trường và phương thức trong thân lớp; và các quan hệ giữa lớp. Với lớp mang khuôn đập vai trò, các trường trong thân lớp được phân tích theo văn phạm 3.1 thành đối tượng quyền có cấu trúc thay vì chuỗi ký tự.

Biểu đồ hoạt động. Mỗi làn bơi (*swimlane*) xác định ca sử dụng hoặc lớp miền mà chuỗi hành động thuộc về; các hành động liên tiếp trong một làn được gom thành một nút hoạt động có thứ tự; cấu trúc rẽ nhánh và rẽ song song được ghi nhận thành nút quyết định tương ứng.

Tập ràng buộc. Mỗi bất biến được phân tích thành cây biểu thức có kiểu, gồm ngữ cảnh, tên bất biến và thân biểu thức.

3.3.3 Vị trí nguồn và chẩn đoán lỗi

Một quyết định thiết kế xuyên suốt: *mỗi phần tử của cây cú pháp mang theo vị trí xuất xứ* – tên tệp và số dòng nơi nó được khai báo. Chi phí là một trường dữ liệu bổ sung trên mỗi nút; lợi ích thì kéo dài tới tận cuối quy trình:

- Bước 4 báo vi phạm bất biến kèm đúng dòng gây lỗi thay vì chỉ nêu tên phần tử;
- Bước 5 chèn được chú thích truy vết vào mã sinh ra, tạo nên quan hệ truy vết hai chiều;
- Người thiết kế sửa lỗi trực tiếp trên tài liệu đặc tả mà không phải dò ngược.

Về phạm vi cú pháp, phương pháp chủ trương hỗ trợ một *tập con được định nghĩa rõ* của ký pháp sơ đồ thay vì cố gắng phủ toàn bộ. Cấu trúc nằm ngoài tập con bị từ chối ngay với thông báo tường minh, thay vì bị bỏ qua âm thầm – lựa chọn này ưu tiên tính tiên đoán được của công cụ hơn là sự dễ dãi.

3.4 Bước 2 - Hợp nhất mô hình

3.4.1 Mô hình miền hợp nhất

Bước 2 gộp ba cây cú pháp thành một cấu trúc duy nhất. Định nghĩa 3.2 phát biểu cấu trúc đó.

Định nghĩa 3.2 (Mô hình miền hợp nhất). Mô hình miền hợp nhất là bộ $\mathcal{M} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A}, \mathcal{I} \rangle$ trong đó $\mathcal{B}$ là tập các ngữ cảnh giới hạn (mỗi ngữ cảnh chứa các lớp miền cùng quan hệ giữa chúng – mối quan tâm cấu trúc), $\mathcal{P}$ là chính sách kiểm soát truy cập theo Định nghĩa 3.1 – mối quan tâm kiểm soát truy cập, $\mathcal{A}$ là tập các đồ thị hoạt động gắn với ca sử dụng – mối quan tâm hành vi, và $\mathcal{I}$ là tập bất biến đọc từ tệp ràng buộc.

Ba khía cạnh nằm trên ba nhánh riêng của cùng một cây, phản ánh trực tiếp yêu cầu về tính trực giao: có thể lược bỏ nhánh $\mathcal{P}$ mà hai nhánh còn lại vẫn là một mô hình miền hoàn chỉnh và vẫn sinh mã được – chỉ là mã không có kiểm soát truy cập.

3.4.2 Tham chiếu chéo giữa các mối quan tâm

Điều làm cho mô hình là *hợp nhất* chứ không phải *đặt cạnh nhau* là các tham chiếu chéo. Có ba loại:

1. **Quyền** → **lớp miền**: thành phần $res(q)$ của mỗi quyền được phân giải từ chuỗi tên thành tham chiếu trực tiếp tới đối tượng lớp miền trong nhánh $\mathcal{B}$.
2. **Hành vi** → **phương thức**: mỗi nút hoạt động được gắn vào ca sử dụng hoặc phương thức tương ứng trong nhánh $\mathcal{B}$.
3. **Bất biến** → **ngữ cảnh**: mỗi bất biến trong $\mathcal{I}$ được gắn vào lớp hoặc vai trò mà nó ràng buộc.

Phép phân giải tham chiếu có thể thất bại – ví dụ quyền trở tới một lớp không tồn tại. Khi đó bước 2 *không* báo lỗi ngay mà ghi nhận tham chiếu ở trạng thái chưa phân giải; việc phán quyết dành cho bước 4, nơi bất biến $\mathcal{I}$ xử lý tình huống này. Lựa chọn này giữ cho mỗi bước có một trách nhiệm duy nhất: bước 2 chỉ hợp nhất, bước 4 chỉ phán quyết.

3.4.3 Bất biến sau hợp nhất

Kế thừa ba tiêu chí của UDML [6], mô hình sau bước 2 phải bảo đảm:

- **Trực giao**: mỗi chú thích kiểm soát truy cập chỉ gắn vào loại phần tử hợp lệ (vai trò gắn ở mức lớp, không gắn ở mức thuộc tính);
- **Gắn kết**: mọi tham chiếu chéo đều phân giải được (kiểm ở bước 4);
- **Truy vết**: mọi phần tử giữ nguyên vị trí xuất xứ nhận được từ bước 1.

3.5 Bước 3 - Làm giàu mô hình

Bước 3 bổ sung vào mô hình những phần tử suy diễn được theo quy ước, để người thiết kế chỉ phải khai báo phần mang tính đặc thù. Có bốn phép biến đổi, trong đó phép thứ tư là trọng tâm của luận văn.

3.5.1 Ba phép biến đổi trên mối quan tâm cấu trúc

(a) Chuẩn hóa lớp miền. Mỗi lớp thực thể chưa có thuộc tính định danh được bổ sung một định danh; các thuộc tính vết thời gian (thời điểm tạo, thời điểm cập nhật) được thêm theo quy ước. Đây là những thuộc tính mà mọi thực thể đều cần nhưng không mang thông tin thiết kế, nên việc bắt người vẽ khai báo chúng chỉ làm sơ đồ rối thêm.

(b) Sinh thao tác mặc định. Gốc tụ hợp không khai báo phương thức nào được xem là chỉ cần các thao tác cơ bản, và nhận đủ bộ thao tác tạo, đọc theo định danh, đọc

danh sách, cập nhật và xóa. Nếu gốc tụ hợp đã khai báo phương thức, phép biến đổi này không can thiệp – nguyên tắc “khai báo tường minh thắng quy ước”.

(c) Sinh kho lưu trữ. Mỗi gốc tụ hợp chưa có kho lưu trữ tương ứng được sinh một giao diện truy xuất, tuân theo khuôn mẫu repository của DDD.

3.5.2 Lan truyền quyền theo phân cấp vai trò

Phép biến đổi thứ tư hiện thực hóa ngữ nghĩa RBAC_1 (Định nghĩa 2.2): tính tập quyền hiệu dụng perm^* cho mọi vai trò. Bài toán không tầm thường vì phân cấp có thể nhiều tầng: quyền của một vai trò ở đáy phải “chảy ngược” lên tất cả các vai trò kế thừa nó, trực tiếp lẫn gián tiếp.

Luận văn sử dụng thuật toán lặp tới điểm bất động, trình bày trong Listing 3.3.

```
Vao : chỉnh sách P = <R, Q, grant, extends, ...>
Ra  : ánh xạ  $\text{perm}^* : R \rightarrow 2^Q$  (tập quyền hiệu dụng)

1  for each r in R do
2     $\text{perm}^*[r] \leftarrow \{ q \mid (r, q) \in \text{grant} \}$       // quyền khai báo trực tiếp
3  end for
4  repeat
5    changed <- false
6    for each (r_con, r_cha) in extends do      // r_con kế thừa r_cha
7      truooc <-  $|\text{perm}^*[r\_con]|$ 
8       $\text{perm}^*[r\_con] \leftarrow \text{perm}^*[r\_con] \cup \text{perm}^*[r\_cha]$ 
9      if  $|\text{perm}^*[r\_con]| > \text{truoc}$  then changed <- true end if
10   end for
11 until changed = false
12 return  $\text{perm}^*$ 
```

Listing 3.3: Thuật toán lan truyền quyền theo phân cấp vai trò

Tính đúng đắn. Thuật toán tính đúng perm^* theo Định nghĩa 2.2. Thật vậy, gọi $\text{perm}^k[r]$ là giá trị của $\text{perm}^*[r]$ sau vòng lặp thứ k . Bằng quy nạp theo k : sau vòng lặp thứ k , $\text{perm}^k[r]$ chứa mọi quyền của các vai trò cách r không quá k bước trên đồ thị kế thừa. Khi vòng lặp dừng ở bước k^* , không còn quyền mới được thêm, nghĩa là mọi vai trò đạt tới được từ r theo quan hệ *extends* đều đã đóng góp quyền của mình – đúng bằng $\bigcup_{r' \preceq r}$ trong định nghĩa.

Tính dừng. Thuật toán luôn kết thúc. Xét đại lượng $\Phi = \sum_{r \in \mathcal{R}} |\text{perm}^*[r]|$. Mỗi vòng lặp hoặc làm Φ tăng thực sự (khi *changed* nhận giá trị đúng), hoặc kết thúc vòng lặp. Vì Φ bị chặn trên bởi $|\mathcal{R}| \times |\mathcal{Q}|$ – một số hữu hạn – nên số vòng lặp không vượt quá $|\mathcal{R}| \times |\mathcal{Q}|$. Đây là lập luận điểm bất động cổ điển trên một dãy hữu hạn với hàm đơn điệu tăng.

Độ phức tạp. Mỗi vòng lặp duyệt qua $|\text{extends}|$ cạnh, mỗi lần hợp tập tốn $O(|\mathcal{Q}|)$. Số vòng lặp bị chặn bởi chiều cao h của đồ thị kế thừa cộng một. Vậy độ phức tạp là $O(h \cdot |\mathcal{R}| \times |\mathcal{Q}|)$.

$|extends| \cdot |Q|$). Trong thực tế h rất nhỏ (thường không quá 3–4 tầng), nên chi phí không đáng kể.

Điều kiện tiên quyết. Lập luận về tính đúng đắn giả định đồ thị kế thừa không có chu trình. Nếu có chu trình, thuật toán vẫn dừng (lập luận về Φ không dùng tới giả thiết này) nhưng kết quả mất ý nghĩa: mọi vai trò trong chu trình sẽ có cùng tập quyền, tức phân cấp trở nên vô nghĩa. Vì vậy bất biến I5 phải được kiểm tra – và trong quy trình, việc này diễn ra ở bước 4, ngay sau bước 3.

Ví dụ 3.1. Xét ba vai trò với quan hệ $Admin \rightarrow Officer \rightarrow Staff$ và các quyền khai báo trực tiếp: $Staff$ có `VIEW_STUDENTS`, $Officer$ có `ENROLL_STUDENTS`, $Admin$ có `MANAGE_STUDENTS`. Sau vòng lặp thứ nhất, $Officer$ nhận thêm `VIEW_STUDENTS` và $Admin$ nhận thêm `ENROLL_STUDENTS`. Sau vòng lặp thứ hai, $Admin$ nhận tiếp `VIEW_STUDENTS` (đến từ $Staff$ qua $Officer$). Vòng lặp thứ ba không thêm gì và thuật toán dừng. Ví dụ này cho thấy vì sao một lượt duyệt là không đủ: quyền của $Staff$ cần hai vòng mới tới được $Admin$.

3.6 Bước 4 - Kiểm chứng ràng buộc

3.6.1 Quy trình đánh giá

Bước 4 đánh giá tập bất biến của Bảng 3.2 trên mô hình đã làm giàu. Với mỗi bất biến thuộc nhóm “thiết kế”, quy trình gồm ba việc: xác định tập đối tượng thuộc ngữ cảnh của bất biến; đánh giá biểu thức trên từng đối tượng; và với mỗi trường hợp sai, tạo một bản ghi chẩn đoán.

Bản ghi chẩn đoán gồm bốn thành phần: mã và tên bất biến bị vi phạm; phần tử mô hình gây vi phạm; vị trí xuất xứ (tệp, dòng) lấy từ thông tin do bước 1 mang theo; và mô tả tình huống vi phạm ở dạng người đọc hiểu được. Thành phần thứ ba là điểm khiến chẩn đoán trở nên hữu dụng trong thực tế: người thiết kế được dẫn thẳng tới dòng cần sửa.

3.6.2 Ranh giới giữa hai thời điểm kiểm chứng

Không phải bất biến nào cũng quyết định được ở mức mô hình. Bảng 3.2 phân định như sau:

- **Kiểm ở thời điểm thiết kế** (I1–I5, và phần tĩnh của I6): các bất biến chỉ phụ thuộc vào bản thân đặc tả. Vi phạm ở nhóm này khiến quy trình dừng, không sinh mã. Đây là giá trị cốt lõi mà phương pháp mang lại so với cách cài đặt thủ công: một chính sách mâu thuẫn bị chặn từ khi còn là bản vẽ.
- **Kiểm ở thời điểm vận hành** (I7, I8, và phần động của I6): các bất biến phụ thuộc trạng thái hệ thống lúc chạy – phiên đăng nhập nào đang mở, bao nhiêu người dùng đang giữ vai trò. Với nhóm này, bước 4 không phán quyết mà *ghi nhận yêu cầu kiểm tra*, để bước 5 sinh chúng thành các điểm kiểm soát trong mã.

Nhận xét 3.3. Ranh giới này không phải là hạn chế kỹ thuật mà là hệ quả logic: một mệnh đề nói về trạng thái phiên làm việc đơn giản là không có giá trị chân lý khi hệ

thống chưa chạy. Điều phương pháp bảo đảm được là: mọi bất biến *quyết định được* ở mức mô hình đều đã được kiểm; và mọi bất biến *không quyết định được* ở mức mô hình đều được sinh thành mã kiểm tra, chứ không bị bỏ quên.

3.7 Bước 5 - Sinh mã theo mẫu

3.7.1 Ánh xạ khái niệm sang sản phẩm mã

Bước 5 biến mô hình hợp lệ thành mã nguồn. Nguyên tắc là ánh xạ có hệ thống: mỗi khái niệm của mô hình tương ứng một loại sản phẩm mã, theo Bảng 3.3.

Bảng 3.3: Ánh xạ khái niệm mô hình sang sản phẩm mã

Khái niệm mô hình	Sản phẩm mã
Ngữ cảnh giới hạn	Một mô-đun bản mẫu phần mềm phân tầng riêng
Lớp thực thể, đối tượng giá trị, sự kiện miền	Lớp mã ở tầng miền
Gốc tụ hợp	Lớp mã ở tầng miền, kèm các thao tác đã làm giàu ở bước 3
Kho lưu trữ	Giao diện ở tầng miền và hiện thực ở tầng hạ tầng
Ca sử dụng (kèm chuỗi hành động AGL)	Đơn vị xử lý ở tầng ứng dụng, thân phương thức phác theo chuỗi hành động
Tập vai trò	Kiểu liệt kê vai trò
Tập quyền	Kiểu liệt kê quyền
Quan hệ gán quyền (sau lan truyền)	Bảng gán quyền tra cứu lúc vận hành
Phân cấp vai trò	Bảng phân cấp phục vụ mở rộng quyền lúc vận hành
Bất biến nhóm vận hành (I6-I8)	Các phép kiểm tra bên trong điểm kiểm soát truy cập
Quyền yêu cầu của một thao tác	Siêu dữ liệu gắn trên điểm cuối tương ứng

3.7.2 Ba nguyên tắc của cơ chế sinh

(a) Mẫu sinh không chứa logic. Mọi tính toán – bao đóng quyền, danh sách phụ thuộc, quy tắc đặt tên – được thực hiện trong một bước chuẩn bị dữ liệu, tạo ra một cấu trúc phẳng chỉ gồm chuỗi và danh sách. Mẫu sinh chỉ điền dữ liệu vào khuôn. Lợi ích: logic sinh mã kiểm thử được bằng kiểm thử đơn vị thông thường, thay vì phải so khớp văn bản kết xuất; và mẫu đủ đơn giản để người không viết ra nó vẫn đọc hiểu.

(b) Chú thích truy vết. Mỗi tệp sinh ra mang một khối chú thích ghi rõ nó được sinh từ phần tử mô hình nào và từ dòng nào của tài liệu đặc tả. Đây là chiều còn lại của quan hệ truy vết hai chiều: từ mô hình xuống mã đã có nhờ quy trình sinh, từ mã ngược lên mô hình có nhờ chú thích này.

(c) Sinh lại an toàn. Bước sinh mã có tính idempotent: với cùng một đặc tả đầu vào, kết quả sinh ra là như nhau ở mọi lần chạy. Nhờ đó việc sinh lại sau khi sửa mô hình tạo ra một khác biệt *đúng bằng* phần đã thay đổi, thuận lợi cho việc rà soát. Đối với phần

mã do lập trình viên bổ sung, phương pháp dùng cơ chế vùng được bảo vệ: các đoạn nằm giữa cặp dấu hiệu quy ước được trích ra trước khi sinh và chèn lại sau khi sinh.

3.8 Các tính chất của phương pháp

Mục này phân tích bốn tính chất mà phương pháp bảo đảm, tương ứng bốn yêu cầu “sao cho” trong phát biểu bài toán ở mục 1.1.3.

Tính trực giao. Mỗi quan tâm kiểm soát truy cập nằm trên một nhánh riêng của mô hình hợp nhất và chỉ liên hệ với mỗi quan tâm cấu trúc qua một loại tham chiếu duy nhất (quyền $\rightarrow$ lớp miền). Hệ quả: loại bỏ toàn bộ nhánh $\mathcal{P}$ khỏi mô hình vẫn cho một mô hình hợp lệ và sinh được mã – chỉ khác là mã không có lớp kiểm soát truy cập. Ở chiều ngược lại, thêm hoặc sửa chính sách không làm thay đổi mã sinh ra cho tầng miền. Tính chất này được kiểm chứng thực nghiệm ở Chương 4.

Tính truy vết hai chiều. Chiều xuôi được bảo đảm bởi bản thân quy trình sinh: mọi phần tử mã đều có nguồn gốc từ một phần tử mô hình. Chiều ngược được bảo đảm bởi chú thích truy vết, dựa trên thông tin vị trí nguồn mà bước 1 gắn vào và các bước sau bảo toàn.

Tính tự động và lặp lại được. Toàn bộ năm bước chạy trong một lượt, không cần can thiệp thủ công ở giữa. Kết hợp với tính idempotent của bước 5, quy trình có thể chạy lại bất kỳ lúc nào; mô hình luôn là nguồn sự thật duy nhất.

Tính độc lập với công nghệ. Bốn bước đầu chỉ thao tác trên các khái niệm mô hình hóa: lớp, quan hệ, vai trò, quyền, bất biến. Tri thức về nền tảng đích tập trung hoàn toàn trong bộ mẫu của bước 5. Do đó, việc chuyển sang một nền tảng đích khác về nguyên tắc chỉ đòi hỏi viết bộ mẫu mới, giữ nguyên bốn bước đầu và toàn bộ siêu mô hình. Cần lưu ý đây là lập luận dựa trên thiết kế; việc kiểm chứng nó bằng thực nghiệm trên một nền tảng thứ hai thuộc hướng phát triển và được nêu lại trong phần phân tích nguy cơ ảnh hưởng tính hợp lệ ở Chương 4.

3.9 Tổng kết chương

Chương này đã trình bày phương pháp đề xuất theo hai phần. Phần thứ nhất định nghĩa ngôn ngữ đặc tả chính sách RBAC với đầy đủ ba thành phần: cú pháp trừu tượng là siêu mô hình bảy khái niệm (Định nghĩa 3.1); cú pháp cụ thể là profile trên biểu đồ lớp với văn phạm quyền có cấu trúc; ngữ nghĩa tĩnh là tám bất biến phân theo ba cấp RBAC. Phần thứ hai trình bày quy trình năm bước, trong đó đáng chú ý là thuật toán lan truyền quyền với chứng minh tính đúng đắn và tính dừng (mục 3.5), và nguyên tắc phân định hai thời điểm kiểm chứng (mục 3.6). Chương tiếp theo trình bày công cụ hiện thực hóa phương pháp và kết quả đánh giá thực nghiệm.

Chương 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Giới thiệu

4.2 Công cụ hỗ trợ

4.3 Ca nghiên cứu CourseMan

4.4 Thiết kế đánh giá

4.5 Kết quả thực nghiệm

4.6 Các nguyên cơ ảnh hưởng tính hợp lệ

4.7 Bàn luận

4.8 Tổng kết chương

Tài liệu tham khảo

- [1] David Basin, Jürgen Doser, and Torsten Lodderstedt. Model driven security: From UML models to access control infrastructures. *ACM Transactions on Software Engineering and Methodology*, 15(1):39-91, 2006. doi: 10.1145/1125808.1125810.
- [2] Duc-Hanh Dang, Duc Minh Le, and Van-Vinh Le. AGL: Incorporating behavioral aspects into domain-driven design. *Information and Software Technology*, 163: 107284, 2023. doi: 10.1016/j.infsof.2023.107284.
- [3] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [4] Martin Fowler. *Domain-Specific Languages*. Addison-Wesley, 2010.
- [5] Martin Gogolla, Fabian Büttner, and Mark Richters. USE: A UML-based specification environment for validating UML and OCL. *Science of Computer Programming*, 69(1-3):27-34, 2007. doi: 10.1016/j.scico.2007.01.013.
- [6] Duc Minh Le and Duc-Hanh Dang. A method for composing concerns into a unified domain model in domain-driven design. In *Proceedings of the International Conference on Knowledge and Systems Engineering (KSE 2025)*. IEEE, 2025. Cần bổ sung tên đồng tác giả, số trang và DOI theo bản kỷ yếu chính thức.
- [7] Duc Minh Le, Duc-Hanh Dang, and Viet-Ha Nguyen. On domain driven design using annotation-based domain specific language. *Computer Languages, Systems & Structures*, 54:199-235, 2018. doi: 10.1016/j.cl.2018.05.001.
- [8] Duc Minh Le, Duc-Hanh Dang, and Viet-Ha Nguyen. Generative software module development for domain-driven design with annotation-based domain specific language. *Information and Software Technology*, 120:106239, 2020. doi: 10.1016/j.infsof.2019.106239.
- [9] Duc Minh Le, Duc-Hanh Dang, and Hieu Dinh Vo. Layered microservices architecture: A multitree-based domain-driven approach. *Information and Software Technology*, 183:107720, 2025. doi: 10.1016/j.infsof.2025.107720.
- [10] Van-Vinh Le, Duc-Hanh Dang, and Duc Minh Le. A pattern-based approach to incorporating behavioral aspects into domain models. In *Proceedings of the 15th International Conference on Knowledge and Systems Engineering (KSE 2023)*. IEEE, 2023. Cần bổ sung số trang và DOI theo bản kỷ yếu chính thức.

- [11] Torsten Lodderstedt, David Basin, and Jürgen Doser. SecureUML: A UML-based modeling language for model-driven security. In *UML 2002 — The Unified Modeling Language*, volume 2460 of *Lecture Notes in Computer Science*, pages 426–441. Springer, 2002. doi: 10.1007/3-540-45800-X_33.
- [12] *NestJS Documentation: Guards, Custom Decorators, Authorization, CQRS*. NestJS, 2025. <https://docs.nestjs.com>.
- [13] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2 edition, 2021.
- [14] *Object Constraint Language (OCL), Version 2.4*. Object Management Group, 2014. Document formal/2014-02-03. <https://www.omg.org/spec/OCL/2.4>.
- [15] *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Object Management Group, 2017. Document formal/2017-12-05. <https://www.omg.org/spec/UML/2.5.1>.
- [16] *PlantUML Language Reference Guide*. PlantUML, 2025. <https://plantuml.com/guide>.
- [17] Indrakshi Ray, Na Li, Robert France, and Dae-Kyoo Kim. Using UML to visualize role-based access control constraints. In *Proceedings of the Ninth ACM Symposium on Access Control Models and Technologies (SACMAT’04)*, pages 115–124. ACM, 2004. doi: 10.1145/990036.990054.
- [18] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [19] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [20] Karsten Sohr, Michael Drouineaud, Gail-Joon Ahn, and Martin Gogolla. Analyzing and managing role-based access control policies. *IEEE Transactions on Knowledge and Data Engineering*, 20(7):924–939, 2008. doi: 10.1109/TKDE.2008.28.
- [21] Vaughn Vernon. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
- [22] Lê Văn Vinh. *Nghiên cứu các kỹ thuật biểu diễn và chuyển đổi mô hình cho thiết kế hướng miền*. Luận án tiến sĩ, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội, 2026.
- [23] Markus Voelter, Sebastian Benz, Christian Dietrich, Birgit Engelmann, Mats Helander, Lennart C. L. Kats, Eelco Visser, and Guido Wachsmuth. *DSL Engineering: Designing, Implementing and Using Domain-Specific Languages*. CreateSpace Independent Publishing Platform, 2013. <http://dslbook.org>.